AhamX Bodhi

Where I Becomes Infinite

Byte Pair Encoding (BPE)

Module 3.3: Tokenization

Prof. Sashikumaar Ganesan | IISc Bengaluru

Session Overview

What You Will Learn

- The original data compression origin of BPE
- The BPE training algorithm step by step
- How the BPE merge table encodes vocabulary
- BPE encoding at inference time
- BPE in GPT-2, GPT-4, and Llama models

Concept at a Glance

- **Duration:** 9 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Algorithm:** Iterative frequency merging
- **Used By:** GPT-2, GPT-4, Llama, Falcon

Learning Objectives

By the End of This Concept You Will Be Able To

- **Explain** the origin of BPE as a data compression algorithm
- **Trace** the BPE training process: initialization, frequency counting, merging
- **Apply** BPE encoding to tokenize a short string using a given merge table
- **Distinguish** BPE from WordPiece and SentencePiece in vocabulary construction
- **Identify** which production LLMs use BPE and how byte-level BPE extends the algorithm

Historical Origin of BPE

From Compression to Tokenization

- BPE was invented in 1994 by Philip Gage as a **lossless data compression** technique
- Original idea: replace the most frequently occurring pair of bytes with an unused byte
- Applied iteratively until no further compression is possible
- Sennrich et al. (2016) adapted BPE for neural machine translation tokenization

The key insight from Sennrich et al.: BPE's merging logic naturally creates a vocabulary of frequent character sequences, which correspond exactly to the morphological units (roots, suffixes, prefixes) that are linguistically meaningful.

The Core Intuition

Why BPE Works for Language

- In any language, some character combinations appear far more often than others
- ``th'', ``ing'', ``tion'' appear millions of times in English text
- If we assign a single token to each frequent combination, sequences become shorter
- Shorter sequences $\Rightarrow$ less computation, faster training, more context per window
- Rare combinations remain as character sequences — no OOV problem

BPE Training: Step 1 -- Initialize

Starting Point

- Begin with a training corpus (all text you want to tokenize)
- Split every word into its individual characters, separated by spaces
- Add an end-of-word marker `</w>` to each word for word boundary awareness
- Example: ``low'' → l o w </w>
- ``lower'' → l o w e r </w>

The initial vocabulary is simply the set of all unique characters in the corpus plus the `</w>` marker. This initial vocabulary is tiny (~300 entries for English) and grows with each merge operation.

$|V| \approx 128K$

200 ~ chord

BPE Training: Step 2 -- Count Pairs

Frequency Counting Example

- Suppose the corpus (with frequencies) is: `low </w>` $\times 5$, `lower </w>` $\times 2$, `newest </w>` $\times 6$, `wider </w>` $\times 3$
- After character splitting, count all adjacent symbol pairs
- Most frequent pair: `(e, r)` appears in `lower` and `wider` = $2 + 3 = 5$ times
- Wait — `(e, s)` in `newest` appears 6 times — that wins

At each step, exactly **one** merge rule is added: the merge that reduces the total number of symbols in the corpus the most. The process is greedy — always take the single best merge available.

BPE Training: Step 3 -- Merge and Repeat

The Merge Loop

- Take the most frequent pair (A, B) and create a new symbol AB
- Replace every occurrence of `A B` with `AB` in the corpus
- Record the merge rule: `A + B` $\rightarrow$ `AB`
- Re-count all adjacent pairs in the updated corpus
- Repeat until the desired vocabulary size V is reached (e.g., 50,000 merges)
- The final vocabulary = initial characters $\cup$ all merged symbols

BPE Training: Worked Example (5 Merges)

Trace on a Tiny Corpus

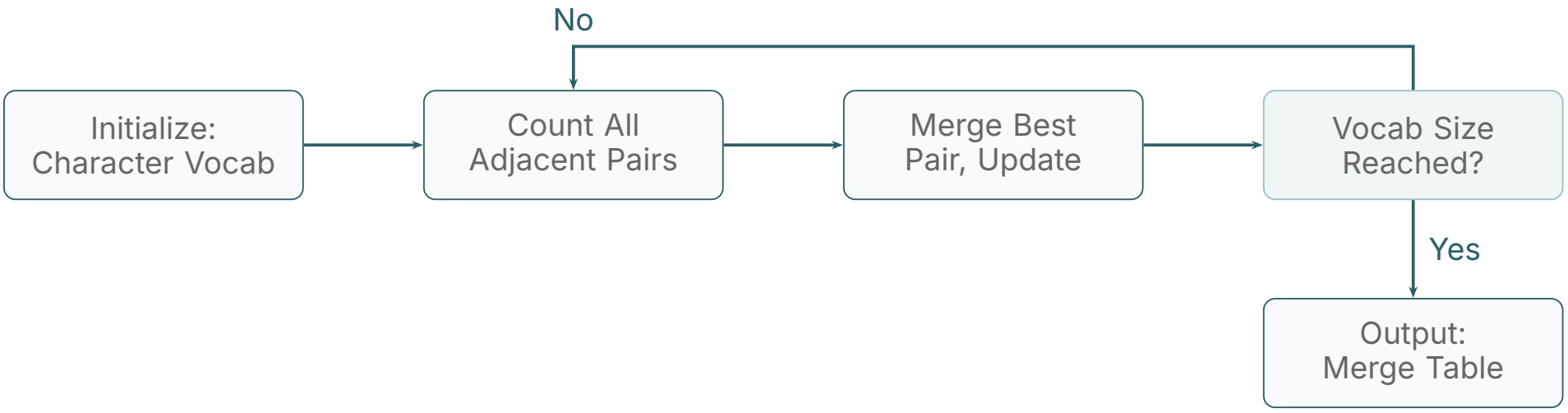
- **Initial:** `l o w </w>`(5), `l o w e r </w>`(2), `n e w e s t </w>`(6), `w i d e r </w>`(3)
- **Merge 1:** `e + s` (freq 6) → `es`. Update: `n ew es t </w>`
- **Merge 2:** `es + t` (freq 6) → `est`. Update: `n ew est </w>`
- **Merge 3:** `l + o` (freq 7) → `lo`. Update: `lo w </w>`(5), `lo w e r </w>`(2)
- **Merge 4:** `lo + w` (freq 7) → `low`. Update: `low </w>`(5), `low e r </w>`(2)
- **Result:** Vocabulary now includes `es`, `est`, `lo`, `low`, plus all characters

BPE Encoding at Inference Time

How New Text is Tokenized

- The merge table from training is fixed and applied in the **same order** it was learned
- Start with the character sequence of the word
- Apply merge rules from highest to lowest frequency until no more rules apply
- Example with learned merges `l+o→lo`, `lo+w→low`: ``lowest'' → `[low, est, </w>]`
- Unknown characters are handled via byte-level fallback (in byte-BPE variants)

BPE Training Flow



The training loop runs until the target vocabulary size is reached. For GPT-2 this meant 50,000 merge operations; for GPT-4 (cl100k), approximately 100,000 merges.

Byte-Level BPE: The GPT-2 Extension

Eliminating the OOV Problem Entirely

- Standard BPE starts from characters; byte-BPE starts from raw **bytes** (256 possible values)
- Any Unicode character encodes as 1–4 bytes in UTF-8
- Therefore any string, in any language, in any encoding, can be tokenized without failure
- GPT-2 introduced byte-level BPE; GPT-4 (cl100k) and Llama continue this approach

Byte-level BPE means the tokenizer has **zero** out-of-vocabulary failures. Emoji, mathematical symbols, code in any programming language, and text in any script are all handled gracefully through byte sequence decomposition.

BPE Properties and Trade-offs

Strengths

- Simple, deterministic algorithm
- Vocabulary size is fully controllable
- No OOV (with byte-level variant)
- Works across all languages
- Fast encoding at inference time

Limitations

- Greedy merging is not globally optimal
- Tokenization is context-independent (same word always splits the same way)
- Sensitive to whitespace pre-tokenization rules
- Different corpora produce different vocabularies

GenAI Application: BPE in GPT Models

OpenAI's Tokenizer Evolution

- **GPT-2:** Introduced byte-level BPE with 50,257 tokens. Whitespace rules: spaces attach to the *following* word
- **GPT-3:** Same tokenizer as GPT-2 for compatibility
- **GPT-3.5 / GPT-4 (cl100k_base):** 100,256 tokens, improved multilingual and code coverage
- **GPT-4o (o200k_base):** 200,019 tokens, further expanded for multimodal contexts

OpenAI's `tiktoken` library implements their BPE tokenizers in Rust with Python bindings, providing tokenization at ~10 million tokens per second on modern hardware.

GenAI Application: BPE in Open-Source LLMs

Meta Llama Tokenizer Series

- **Llama 1 and Llama 2:** Used SentencePiece with BPE algorithm, 32,000 tokens. Limited multilingual performance
- **Llama 3 (2024):** Switched to tiktoken-style byte-level BPE, 128,256 vocabulary. Dramatically improved multilingual token efficiency
- **Falcon:** Custom BPE tokenizer, 65,024 tokens with multilingual focus
- **Mistral:** Llama 2 SentencePiece BPE, 32,000 tokens; compatibility with Llama ecosystem

The jump from 32K to 128K vocabulary in Llama 3 reduced Hindi and Arabic token fertility by approximately 3x, directly improving model performance on non-English benchmarks.

GenAI Application: BPE Tokenizer Libraries

Production Tools for BPE

- **tiktoken (OpenAI):** Rust-core BPE library, used for all GPT models. Importable as a Python package
- **HuggingFace Tokenizers:** Rust-core library supporting BPE, WordPiece, Unigram. Powers all HuggingFace model cards
- **SentencePiece (Google):** C++ library supporting BPE and Unigram algorithms, language-agnostic
- **tokenizers.BpeTrainer:** Train custom BPE vocabularies from any text corpus in under a minute

BPE vs WordPiece: Key Differences

Property	BPE	WordPiece
Merge Criterion	Highest raw frequency	Highest likelihood gain
Training Objective	Data compression	Language model probability
Result Bias	Favours very common bigrams	Favours linguistically meaningful units
OOV Handling	Byte-level fallback	`[UNK]` or byte fallback
Key Users	GPT-2/4, Llama 3, Falcon	BERT, DistilBERT, Electra

Common Misconceptions About BPE

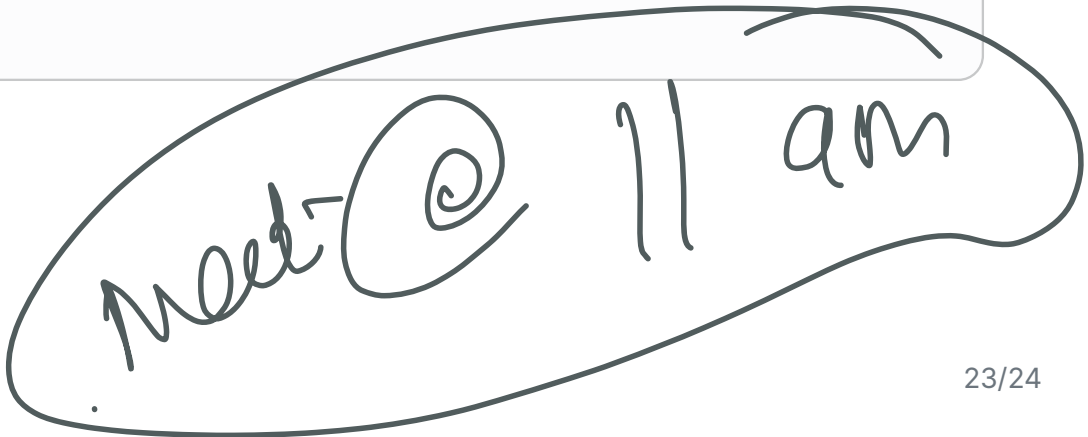
Myths and Corrections

- **Myth:** “BPE merges the most frequent words.” BPE merges the most frequent *adjacent pairs*, which can be any sub-string fragment
- **Myth:** “BPE is the same as WordPiece.” They use different scoring criteria and produce different vocabularies
- **Myth:** “A larger vocabulary is always better.” Larger vocabularies require larger embedding matrices and more data to learn each token well
- **Myth:** “BPE produces consistent tokenizations across models.” Different merge tables produce completely different tokenizations of the same input

Key Takeaways

What You Have Learned

- BPE is a greedy iterative algorithm that merges the most frequent adjacent symbol pairs
- Training produces a fixed **merge table** that encodes the entire vocabulary construction history
- Encoding at inference applies merge rules in training order to any input string
- Byte-level BPE (GPT-2+, Llama 3) eliminates OOV by operating on raw UTF-8 bytes
- BPE powers the most widely deployed LLMs today: GPT-4, Llama 3, Falcon, Mistral
- The algorithm is simple, deterministic, and fast — qualities that make it production-ready



Thank You

Byte Pair Encoding (BPE) — Module 3.3